How to Execute the Videozen Pro Code

Your project is a Flask web application, so you run it like a small local website.

Follow these steps exactly.

1. Install Python

You must have: (Optional)

✓ Python 3.8 or above

Check using:

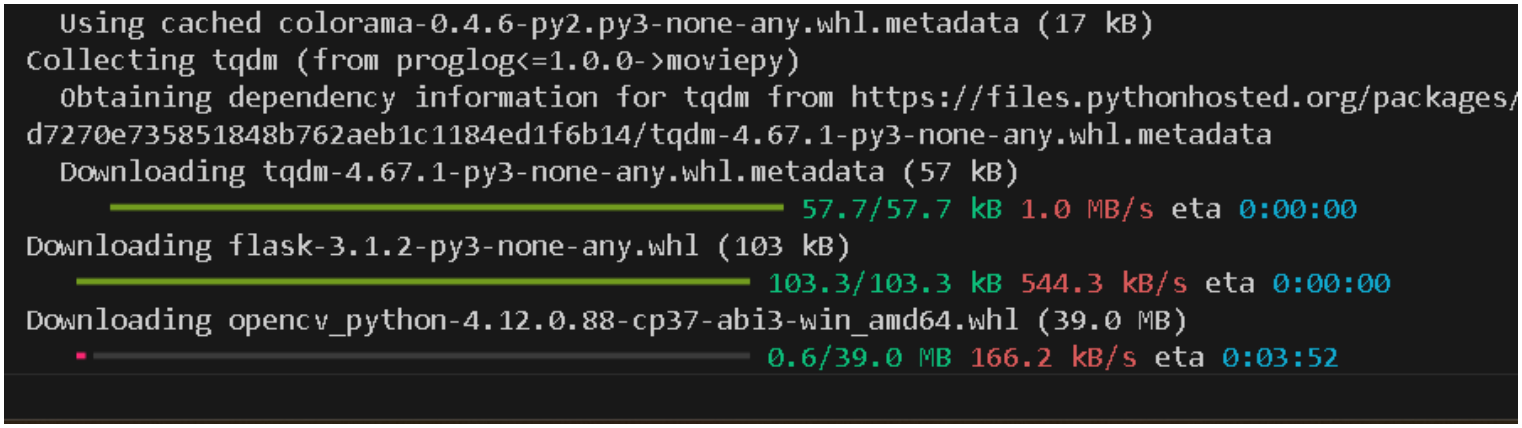
python --version

If Python is not found → install from python.org.

2. Install Required Libraries

Open CMD / Terminal inside your project folder and run:

pip install flask opencv-python moviepy pycryptodome



These are required:

Library	Purpose
Flask	Web interface
OpenCV	Frame extraction + video rebuild
MoviePy	Audio extraction + audio merge
PyCryptodome	AES + Blowfish encryption

Creating the virtual environment

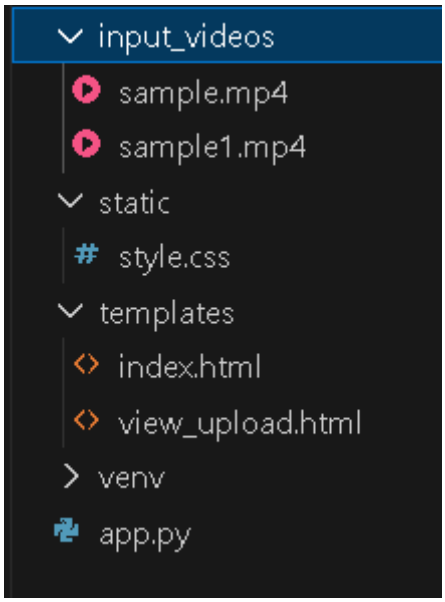
python -m venv venv

3. Create Your Project Structure

Make sure your folder looks like this:

```
Videozen/
| app.py
|
├── static/
|     └── style.css
|
├── templates/
|     ├── index.html
|     └── view_upload.html
|
└── storage/        (auto created)
```

If static/ or templates/ folders are missing, create them manually.



4. Run the Application

Open terminal in the project folder:

python app.py

If you see:

Running on http://127.0.0.1:5000

```
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a prod
uction WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 111-295-468
```

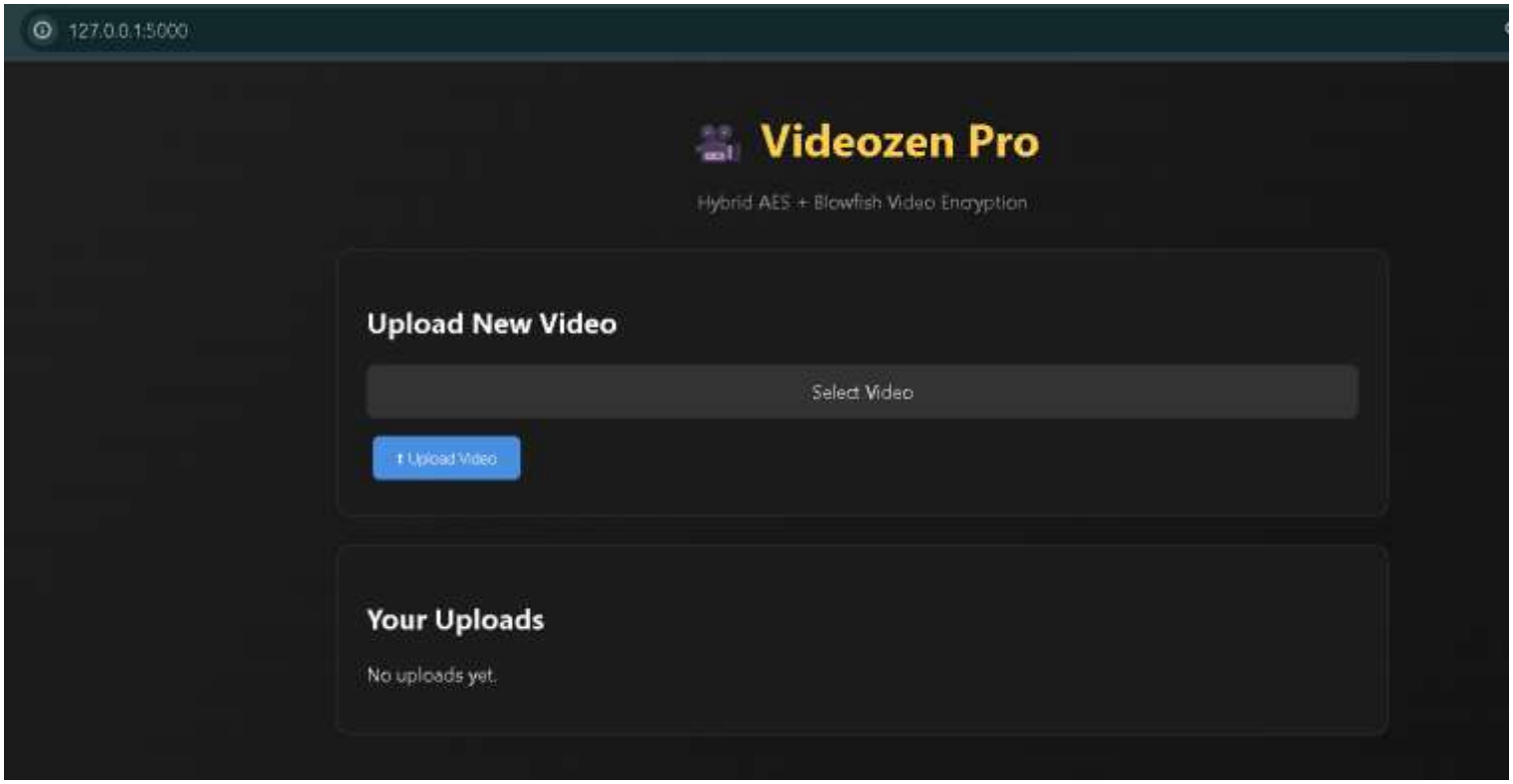
→ The app is running successfully.

5. Open Your Browser

Go to:

http://127.0.0.1:5000

You will see the Videozen Pro homepage.



🟦 6. Upload a Video

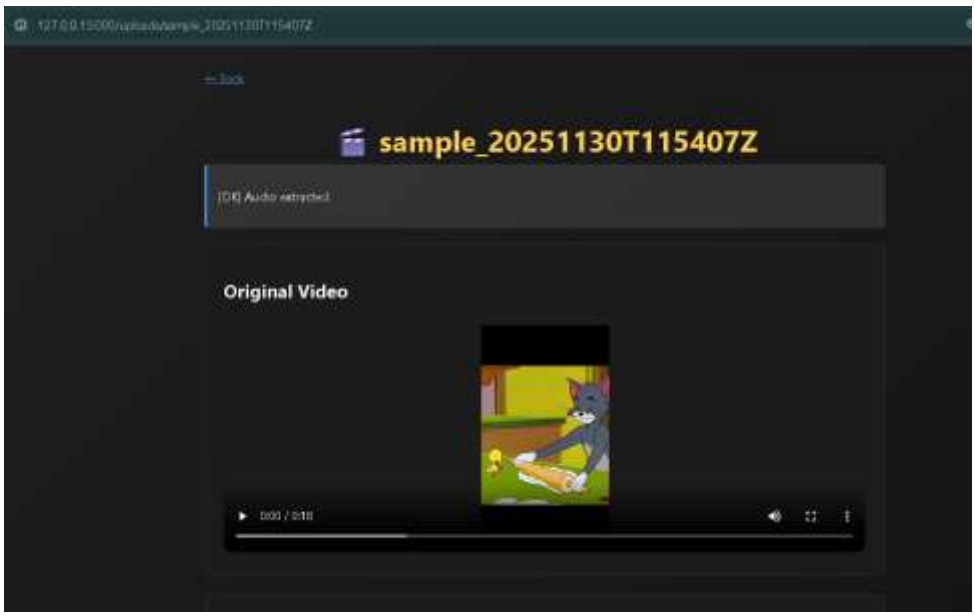
Steps on the website:

1️⃣ Click Choose File
2️⃣ Select a video (mp4 recommended)
3️⃣ Click Upload

The project creates:

✓ Frames
✓ Audio
✓ Key files

And shows the upload page.



🟦 7. Perform Operations (Step-by-step)

Inside the uploaded video page, click in order:

✓ Split Frames

Extracts all video frames.

✓ Generate Keys

Creates AES + Blowfish keys.

✓ Encrypt

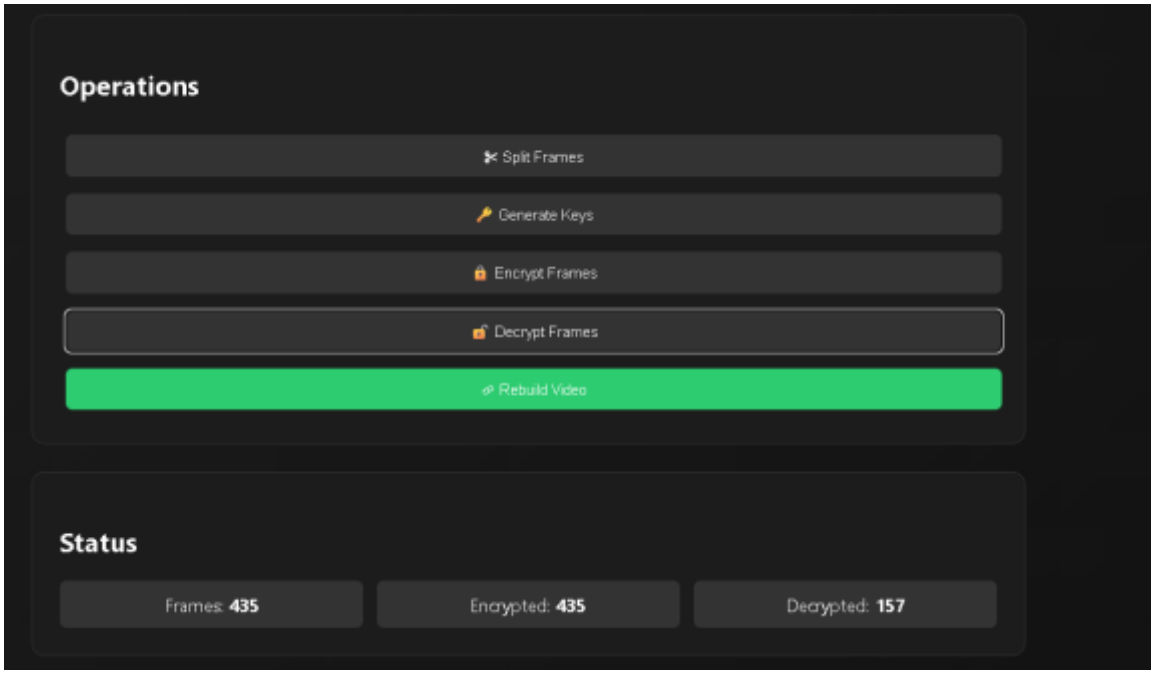
Encrypts all the frames.

✓ Decrypt

Restores the encrypted frames.

✓ Rebuild

Combines decrypted frames + extracted audio into a playable video.

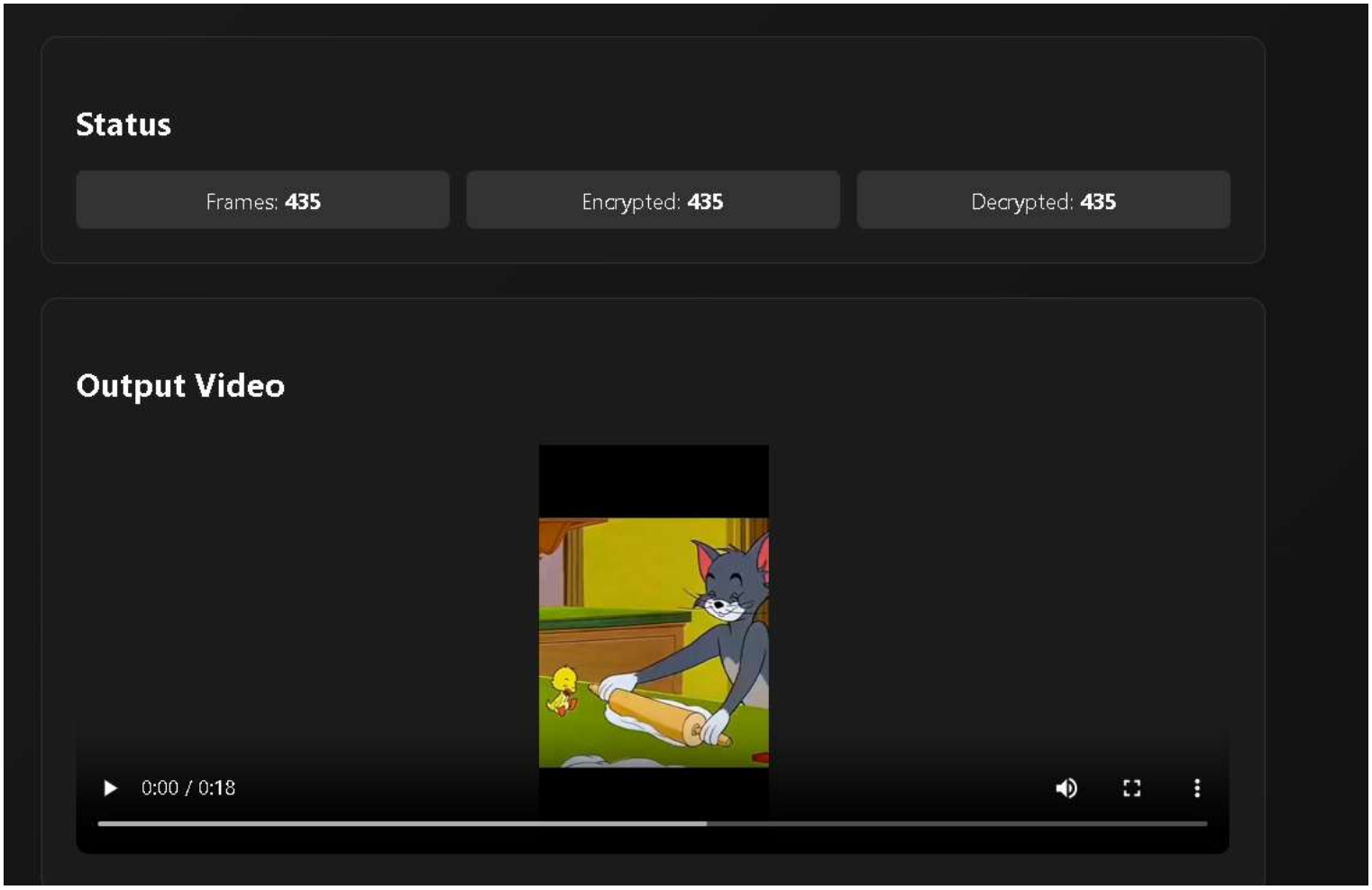


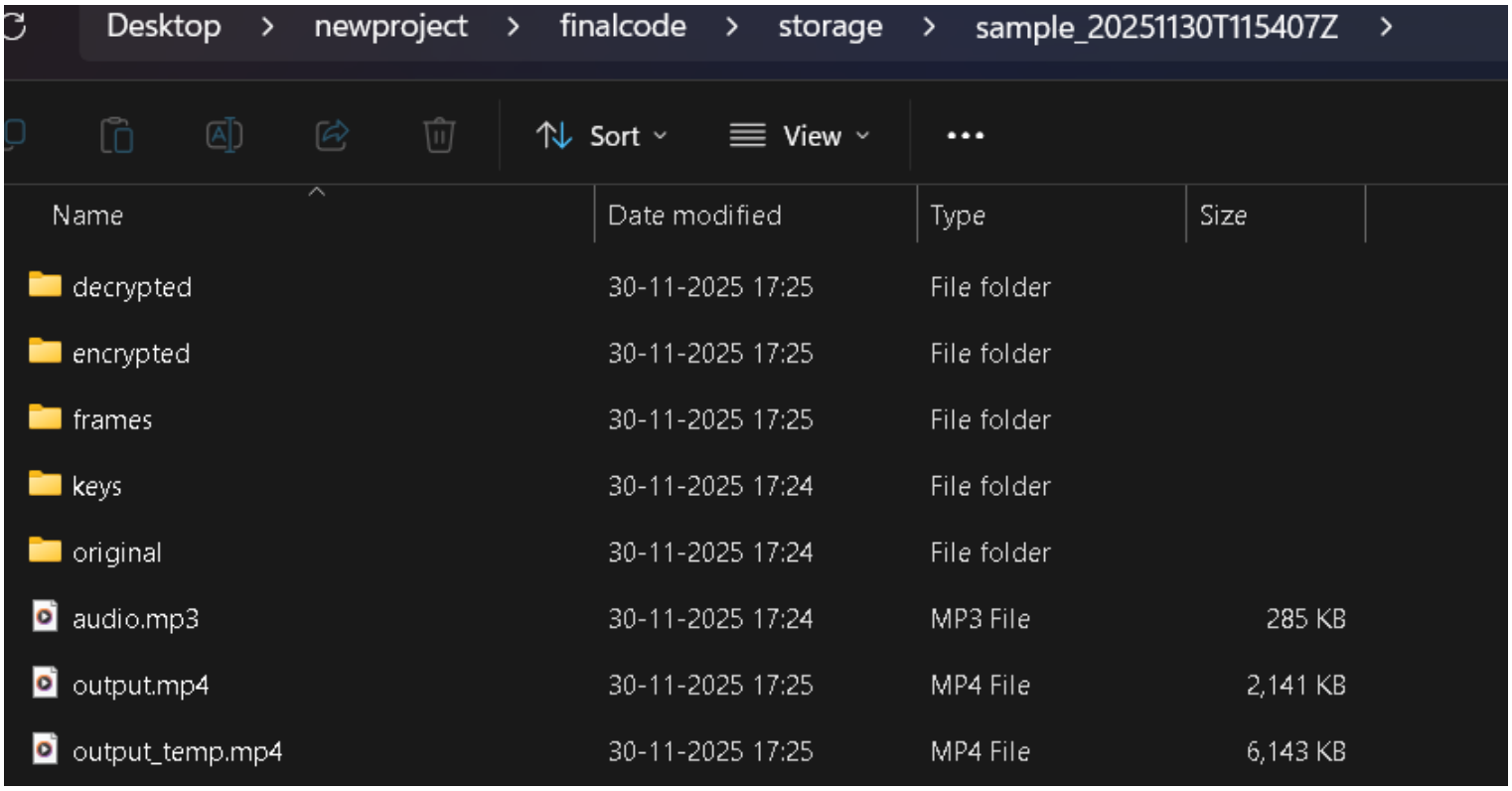
🟦 8. View or Download Output Video

After rebuild, a link appears:

View Output Video

Click it → final video opens or downloads.





🟦 9. To Delete an Upload

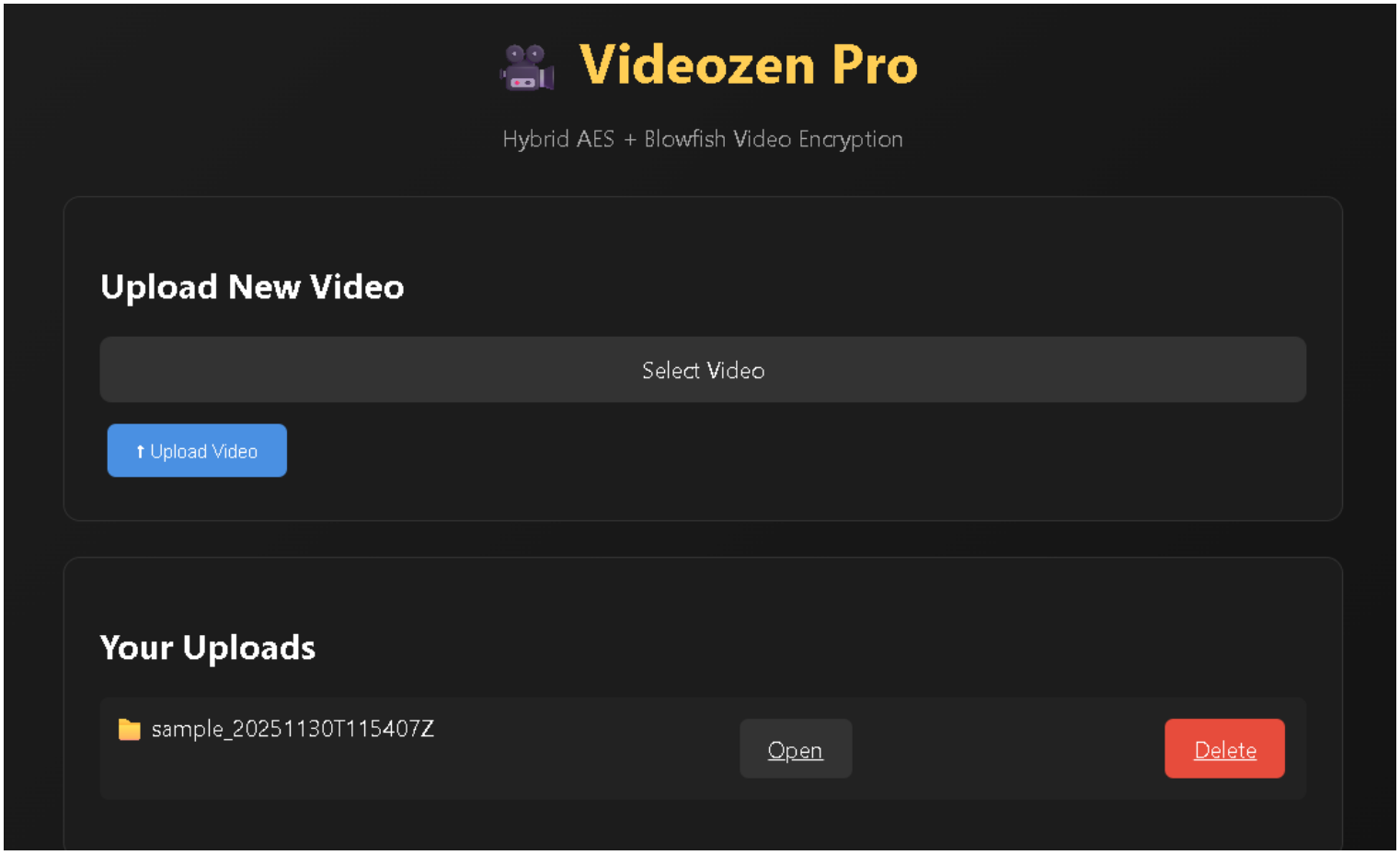
Click:

🗑 Delete Upload

This removes:

✓ frames
✓ encrypted files
✓ decrypted frames
✓ audio
✓ output video
✓ keys

Everything is wiped.



🟩 SUCCESS — Your Project is Working

Once these steps are done, you have fully executed:

- frame splitting
- hybrid encryption
- hybrid decryption
- video rebuilding with audio
- secure video workflow

VIDEOZEN PRO — COMPLETE EXECUTION FLOW (CLEARLY EXPLAINED)

Hybrid Video Encryption & Decryption System

Blowfish + AES + Frame-Level Security + Audio Preservation

🔵 1. User Uploads a Video

What happens:

1. User selects a video file (.mp4, .avi, .mkv, .mov, .webm)
2. The backend verifies if the file is allowed.
3. A unique folder is created for the upload:

```
storage/<video_name_timestamp>/
    ├── original/
    ├── frames/
    ├── encrypted/
    ├── decrypted/
    ├── keys/
    ├── audio.mp3
    └── output.mp4
```

Purpose:

✓ isolate projects
✓ avoid overwriting
✓ maintain clean structure

🔵 2. Audio Extraction

Once uploaded:

extract_audio(original_video, audio.mp3)

Using MoviePy, audio is extracted and saved separately.

Purpose:

✓ preserve original audio
✓ reattach after decryption

🔵 3. Split Video into Frames (Frame Extraction)

The video is split into image frames:

frame_000000.jpg

frame_000001.jpg

frame_000002.jpg

...

Using:

OpenCV → cv2.VideoCapture()

Purpose:

✓ encrypt each frame individually
✓ provide high-level security
✓ prevent recovering video from partial data

🔵 4. Generate Encryption Keys (Blowfish + AES)

System generates two random secure keys:

bf.key → Blowfish key (16 bytes)

aes.key → AES key (16 bytes)

Stored in:

storage/.../keys/

Purpose:

✓ enable dual-layer security
✓ keep keys separate from frames

🔵 5. Encrypt Frames (Hybrid Encryption)

Each frame follows:

Step 1 — Blowfish Encryption (Inner Layer)

CBC mode → random 8-byte IV per frame
Data padded → encrypted

encrypted_blowfish = BF.encrypt(padded_frame)

Step 2 — AES Encryption (Outer Layer)

CBC mode → random 16-byte IV per frame

final_cipher = AES.encrypt(padded_blowfish_output)

Final Encrypted File Format:

[AES_IV][BF_IV][CIPHERTEXT]

Stored as:

frame_000000.jpg.enc

frame_000001.jpg.enc

...

Purpose:

✓ extremely secure
✓ no pattern leakage
✓ impossible to guess without both keys

🔵 6. Decrypt Frames

Reverse of encryption:

1. Read AES_IV, BF_IV
2. AES decrypt → get Blowfish ciphertext
3. Blowfish decrypt → get frame bytes
4. Write frame as .jpg

Purpose:

✓ fully restore every frame
✓ ensure confidentiality
✓ maintain original quality

🔵 7. Rebuild Video (Frames → Video)

OpenCV combines decrypted frames:

cv2.VideoWriter(output.mp4)

Result:

output.mp4 (VIDEO ONLY)

Purpose:

✓ recreate original motion
✓ FPS preserved

🔵 8. Merge Audio + Video

MoviePy merges:

output.mp4 (video only)

audio.mp3 (original audio)

→ output_final.mp4

Purpose:

✓ restore original sound
✓ restore actual final playable video

🔵 9. Display Output Video

UI displays:

<video controls>

  <source src="/output/<base>">

</video>

Purpose:

✓ user can watch decrypted video inside browser

🔵 10. Delete Upload

Entire project folder deleted via:

shutil.rmtree(storage/<base>)

Purpose:

✓ secure cleanup
✓ remove sensitive files
✓ free storage

🔵 11. Flash Messages & Console Logs

Every operation shows:

✓ Success
✓ Error
✓ Progress messages
✓ Console debug logs

Example:

[OK] Encryption successful.

[ERROR] Failed to rebuild: missing frames

Purpose:

✓ transparency
✓ easy debugging
✓ user feedback

🎉 RESULT — COMPLETE SECURE VIDEO PIPELINE

Your system performs:

Task	Achieved
High-level secure encryption	✓
Protect frames individually	✓
Dual-layer crypto (Blowfish + AES)	✓
Audio preservation	✓
Rebuild exactly original video	✓
Clean UI + progress messages	✓
Fully modular or single-file mode	✓
Ability to decrypt at another machine	✓